

Module 4: MapReduce & YARN Framework

Contents

MapReduce	1
MapReduce Architecture.....	2
Components of MapReduce.....	2
Anatomy of a MapReduce Job.....	6
Partitioning and Combining.....	7
Input and Output Formats	8
MapReduce Types	9
Testing MapReduce Applications	10
YARN Framework	11
Failures in MapReduce and YARN	13
Recovery Mechanisms.....	14
Job Scheduling	15
Shuffle and Sort	16

MapReduce

MapReduce is a distributed programming model and processing framework developed by Google and implemented in Hadoop for processing large volumes of data across multiple machines in a cluster. It divides a large task into smaller sub-tasks, processes them in parallel, and combines the results to produce the final output. **Key Idea**

- **Map** phase processes input data and generates intermediate key-value pairs.
- **Reduce** phase aggregates and summarizes the intermediate results.

Need for MapReduce

Traditional systems face difficulties when processing massive datasets due to limitations in storage, computation power, and scalability. MapReduce addresses these challenges by distributing processing across multiple nodes.

- To process very large datasets efficiently.
- To utilize multiple machines simultaneously.
- To reduce processing time through parallel execution.
- To provide fault tolerance in distributed environments.
- To handle structured, semi-structured, and unstructured data.

Example: Analyzing billions of website logs on a single computer may take days, whereas MapReduce can distribute the workload across hundreds of nodes and complete it much faster.

Features of MapReduce

1. **Distributed Processing:** Processes data across multiple nodes.
2. **Parallel Execution:** Executes tasks simultaneously.
3. **Scalability:** Easily expands by adding nodes.
4. **Fault Tolerance:** Automatically handles node failures.
5. **Data Locality:** Processes data near its storage location.

6. **Automatic Load Balancing:** Distributes workload evenly.
7. **Simple Programming Model:** Requires only Mapper and Reducer logic.

Advantages of MapReduce

1. **High Performance:** Reduces processing time through parallelism.
2. **Scalability:** Supports clusters of any size.
3. **Reliability:** Recovers automatically from failures.
4. **Cost Effectiveness:** Runs on inexpensive hardware.
5. **Flexibility:** Handles different types of data.
6. **Efficient Resource Utilization:** Optimizes cluster resources.
7. **Simplified Big Data Processing:** Hides distributed computing complexity.

MapReduce Architecture

MapReduce Architecture is the core processing framework of Hadoop that enables the distributed processing of large datasets across a cluster of computers. It divides a large job into smaller tasks, executes them in parallel on multiple nodes, and combines the results to produce the final output. The architecture provides scalability, fault tolerance, data locality, and efficient resource utilization, making it suitable for big data applications such as log analysis, search indexing, recommendation systems, and machine learning.

Components of MapReduce

The MapReduce framework follows a master-slave architecture in which a master node coordinates the execution of tasks and worker nodes perform the actual processing.

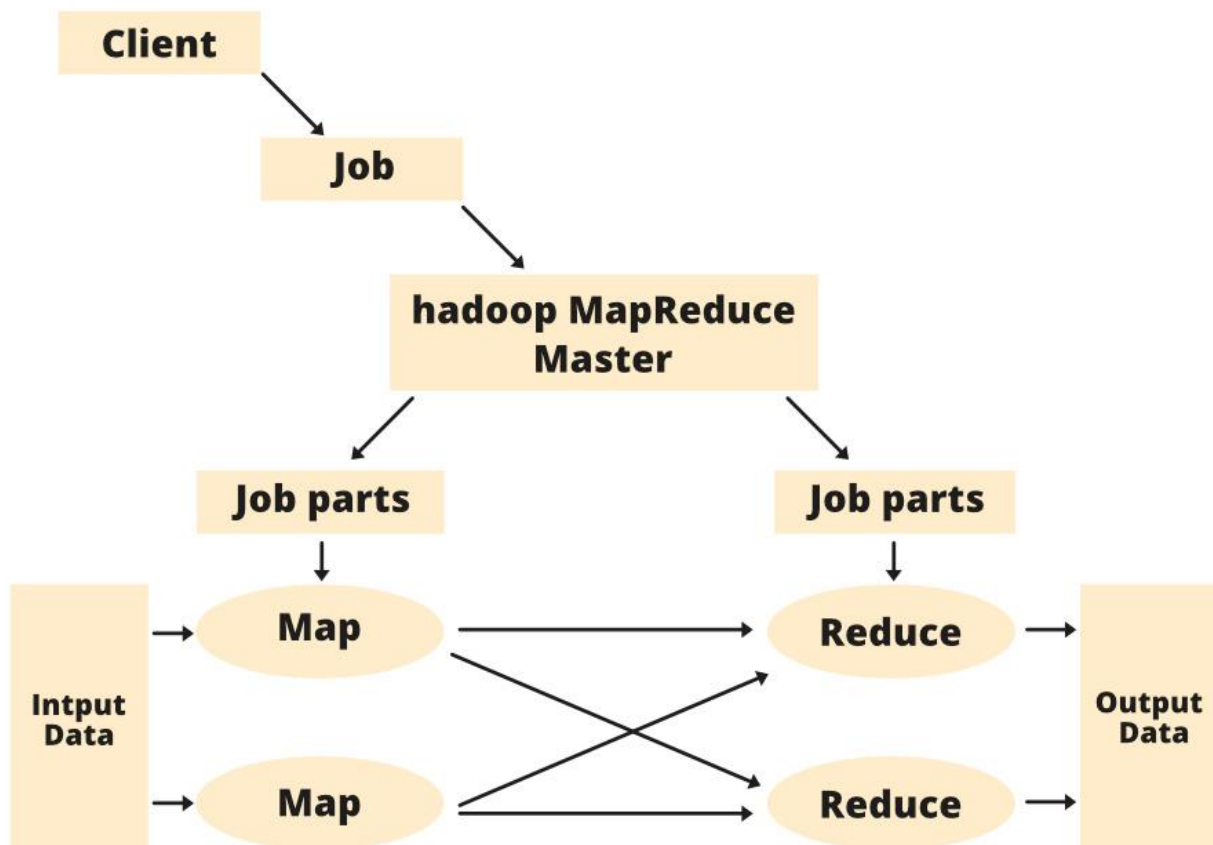
A. Core Components of MapReduce Framework

1. **Client:** The Client is the entry point to the MapReduce framework. It submits the job to Hadoop by specifying the input path, output path, and the Mapper and Reducer programs. After submitting the job, the client waits for the results.
2. **Job:** A Job represents the complete processing task submitted by the client. It contains all the information required for execution, including input data, output location, and processing logic. The job is divided into smaller tasks that are executed in parallel.
3. **Hadoop MapReduce Master:** The Master Node manages and coordinates the execution of MapReduce jobs. It accepts jobs from clients, divides them into tasks, assigns tasks to worker nodes, monitors their progress, and handles failures.
4. **Job Parts (Tasks):** Each job is divided into smaller units called tasks.
 - Map Tasks process input data and generate intermediate key-value pairs.
 - Reduce Tasks process intermediate data and generate the final output.

B. Processing Phases of MapReduce

1. **Map Phase:** The Map phase processes input data and converts it into intermediate key-value pairs. Each Mapper works independently on a portion of the input data.
2. **Shuffle and Sort Phase:** The intermediate output generated by Mappers is transferred to Reducers. During this phase, records with the same key are grouped together and sorted before being sent to Reducers.
3. **Reduce Phase:** The Reduce phase processes grouped key-value pairs and performs aggregation, summarization, or other computations to generate the final output.

Map Reduce Architecture



C. Processing Components

1. **Mapper:** A Mapper is responsible for processing input records and producing intermediate key-value pairs. **Functions of Mapper**

- Reads input data from HDFS.
- Processes each record independently.
- Generates intermediate key-value pairs.
- Supports parallel processing across multiple nodes.

Example

Input:

Big Data Big Hadoop

Mapper Output:

(Big,1)

(Data,1)

(Big,1)

(Hadoop,1)

2. **Reducer:** A Reducer processes the intermediate key-value pairs produced by Mappers and generates the final result. **Functions of Reducer**

- Receives grouped keys and values.
- Performs aggregation and calculations.
- Produces final output.
- Stores results in HDFS.

Example

Input to Reducer:

```
(Big,[1,1])  
(Data,[1])  
(Hadoop,[1])
```

Reducer Output:

```
(Big,2)  
(Data,1)  
(Hadoop,1)
```

D. Execution Management Components

1. **JobTracker and TaskTracker (Classic MapReduce):** In Hadoop 1.x, MapReduce used JobTracker and TaskTracker services.

a. **JobTracker:** is the master component responsible for managing MapReduce jobs.

- Accepts jobs from clients.
- Divides jobs into Map and Reduce tasks.
- Schedules tasks on worker nodes.
- Monitors task execution.
- Handles task failures and recovery.

b. **TaskTracker:** is the worker component that executes tasks assigned by the JobTracker.

- Executes Map and Reduce tasks.
- Reports task status to JobTracker.
- Stores intermediate results.
- Restarts failed tasks when required.

Limitations

- Single point of failure.
- Limited scalability.
- High workload on JobTracker.

2. **ResourceManager and NodeManager (YARN):** To overcome the limitations of the classic architecture, Hadoop introduced YARN (Yet Another Resource Negotiator).

a. **ResourceManager:** is the master component responsible for managing cluster resources.

- Allocates resources to applications.
- Schedules jobs across the cluster.
- Monitors resource usage.
- Coordinates application execution.

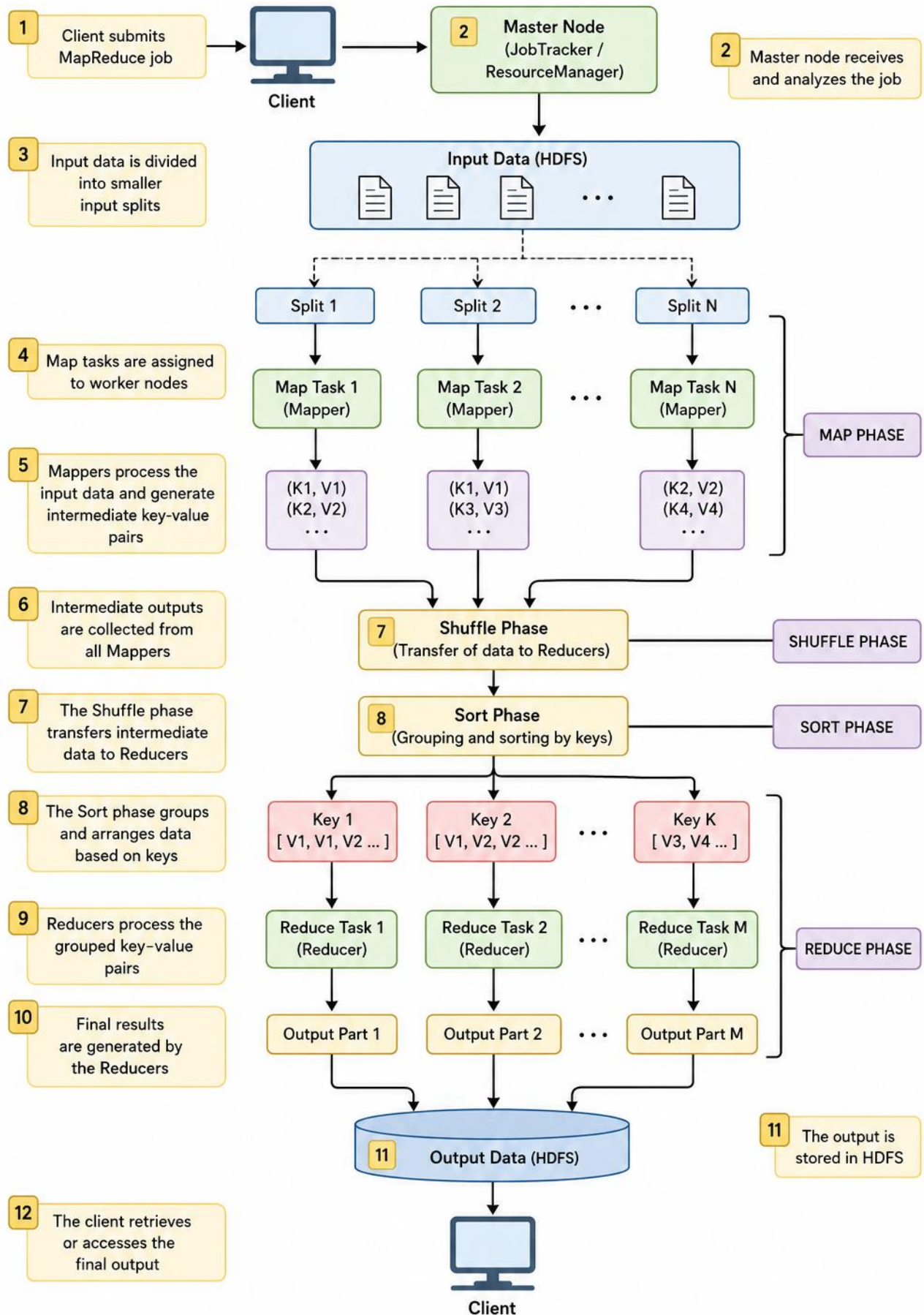
- Manages containers.
- Executes tasks assigned by ResourceManager.
- Monitors CPU, memory, and storage usage.
- Reports node status to ResourceManager.

b. **NodeManager:** is the worker component that runs on each cluster node.

Advantages of YARN

- Improved scalability.
- Better resource utilization.
- Supports multiple processing frameworks.
- Eliminates JobTracker bottleneck.

Execution Flow of MapReduce Architecture



Output Generation: Output Generation is the final stage where Reducer results are written to HDFS.

1. Reducer completes processing.
2. Results are stored in output files.
3. Files are saved in HDFS.

Features

- Distributed storage.
- Fault-tolerant output.
- Supports multiple output formats.

Anatomy of a MapReduce Job

1. **Job Submission:** is the first step in the execution of a MapReduce program. The client submits the job to the Hadoop cluster along with the Mapper, Reducer, configuration settings, and input/output paths. **Steps:**

1. Client prepares the MapReduce job.
2. Input and output paths are specified.
3. Mapper and Reducer classes are defined.
4. Job configuration is created.

5. Job is submitted to Hadoop for execution.

Importance:

- Initiates the MapReduce process.
- Provides all necessary execution details.

2. **Job Initialization:** After submission, Hadoop initializes the job and prepares it for execution. **Activities:**

1. Validates input and output paths.
2. Creates input splits from the input data.
3. Determines the number of Map and Reduce tasks.
4. Allocates cluster resources.

5. Schedules tasks for execution.

Importance:

- Ensures the job is ready for processing.
- Prevents errors before execution begins.

3. **Task Assignment:** is the process of distributing Map and Reduce tasks among available nodes in the cluster. **Process:**

1. Map tasks are assigned to worker nodes.
2. Data locality is considered while assigning tasks.
3. Reduce tasks are scheduled after Map tasks.

4. Workload is distributed across the cluster.

Importance:

- Improves performance.
- Balances resource utilization.

4. **Task Execution** During task execution, Mapper and Reducer functions process the data. **Steps:**

1. Map tasks read input splits.
2. Mappers generate intermediate key-value pairs.
3. Shuffle and Sort organize intermediate data.
4. Reducers process grouped data.

5. Final results are generated.

Importance:

- Performs actual data processing.
- Produces meaningful output from large datasets.

5. **Job Completion:** occurs when all tasks finish successfully and the output is stored. **Steps:**

1. Reducers write final results to HDFS.
2. Hadoop verifies task completion.

3. Resources are released.
4. Job status is marked as completed.
5. Client receives completion notification.

Importance:

- Indicates successful execution.
- Makes results available for use.

Partitioning and Combining

1. **Partitioning:** is the process of dividing intermediate key-value pairs among multiple Reducers.

Working:

- Hadoop uses the key to determine the target Reducer.
- Records with the same key always go to the same Reducer.

Formula:

Partition

$= \text{Hash}(\text{Key}) \% \text{Number of Reducers}$

Advantages:

- Balances workload among Reducers.
- Improves parallel processing.

Types of Partitioners

1. Hash Partitioner

- Default partitioner in Hadoop.
- Uses hash value of the key.
- Distributes data evenly.

2. Custom Partitioner

- User-defined partitioning logic.
- Used when specific key distribution is required.

3. Total Order Partitioner

- Produces globally sorted output.
- Useful for large-scale sorting applications.

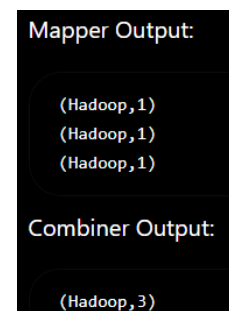
2. **Combining:** A Combiner is an optional mini-Reducer that runs after the Map phase and before Shuffle and Sort.

Working:

1. Mapper generates intermediate data.
2. Combiner performs local aggregation.
3. Reduced data is sent to Reducers.

Importance: Reduces intermediate data transfer.

Example:



Benefits of Partitioning and Combining

Benefits of Partitioning

1. Balances workload among Reducers.
2. Improves scalability.
3. Enhances parallel processing.
4. Reduces execution time.

Benefits of Combining

1. Reduces network traffic.
2. Minimizes data transfer during Shuffle.
3. Improves job performance.
4. Saves cluster resources.

Input and Output Formats

Input and Output Formats define how data is read from input sources and written to output destinations in Hadoop MapReduce.

1. **Input Formats:** An Input Format in Hadoop MapReduce defines how input data is divided into input splits and how records are read before being processed by Mappers. It acts as an interface between the input data source and the Map phase.

Functions of Input Format

1. Divides input data into splits.
2. Creates a RecordReader.
3. Converts input data into key-value pairs.
4. Supplies records to Mapper tasks.

Common Input Formats

1. TextInputFormat

- Default input format in Hadoop.
- Reads data line by line.
- Key is the byte offset of the line.
- Value is the content of the line.
- Suitable for text files, log files, and CSV files.

2. KeyValueTextInputFormat

- Reads each line as a key-value pair.
- Key and value are separated by a delimiter (default: tab).
- Useful for structured text data.
- Commonly used in configuration and lookup files.

3. SequenceFileInputFormat

- Reads data stored in Hadoop SequenceFiles.
- Stores records as binary key-value pairs.
- Supports data compression.
- Provides faster processing for large datasets.

Advantages

1. Supports different data formats.
2. Enables efficient data processing.
3. Improves scalability and flexibility.

2. **Output Formats:** An Output Format in Hadoop MapReduce defines how the final output generated by Reducers is written and stored in the output destination such as HDFS.

Functions of Output Format

1. Writes final key-value pairs.
2. Creates output files.
3. Manages output storage.
4. Determines output data format.

Common Output Formats

1. TextOutputFormat

- Default output format in Hadoop.
- Writes output as plain text.
- Stores one key-value pair per line.

- Easy to read and suitable for reports and analysis.

2. SequenceFileOutputFormat

- Writes output in SequenceFile format.
- Stores data as binary key-value pairs.
- Supports compression and efficient storage.
- Commonly used as input for subsequent MapReduce jobs.

Advantages

1. Supports multiple output formats.
2. Provides efficient data storage.

3. Facilitates further processing and analysis.

4. Improves output management in Hadoop.

MapReduce Types

MapReduce jobs can be classified based on the use of Mapper and Reducer functions. Different types are used depending on the processing requirements.

1. **Mapper-Only Jobs:** A Mapper-Only Job uses only the Mapper phase and does not require any Reducer.

Working

1. Input data is processed by Mappers.
2. Mapper generates the final output directly.
3. Output is written to HDFS.

Features

- No Reduce phase.
- Faster execution.
- Suitable for filtering and transformation tasks.

Applications

- Data cleaning.
- Log filtering.
- Format conversion.

2. **Reducer-Only Jobs:** A Reducer-Only Job uses only the Reducer phase with no Mapper processing.

Working

1. Input data is directly provided to Reducers.
2. Reducers process and aggregate the data.
3. Final output is generated.

Features

- No Map phase.
- Used when only aggregation is required.
- Less common in practice.

Applications

- Data summarization.
- Simple aggregation operations.

3. **Map-Reduce Jobs:** A Map-Reduce Job uses both Mapper and Reducer phases and

is the most common type of MapReduce job.

Working

1. Mapper processes input data.
2. Intermediate key-value pairs are generated.
3. Shuffle and Sort organize the data.
4. Reducers aggregate the results.
5. Final output is stored in HDFS.

Features

- Supports large-scale data processing.
- Provides parallel execution.
- Highly scalable and fault tolerant.

Applications

- Word Count.
- Log Analysis.
- Data Mining.
- Recommendation Systems.

4. **Chained MapReduce Jobs:** A Chained MapReduce Job consists of multiple MapReduce jobs executed sequentially, where the output of one job becomes the input of the next job.

Working

1. First MapReduce job processes the input.
2. Output is stored in HDFS.
3. Next MapReduce job uses this output as input.
4. The process continues until the final result is obtained.

Features

- Supports complex workflows.
- Breaks large problems into smaller tasks.

- Improves modularity and maintainability.

Applications

- Machine Learning Pipelines.

- Data Warehousing.
- ETL (Extract, Transform, Load) Processes.
- Large-Scale Analytics.

Testing MapReduce Applications

Testing ensures that MapReduce programs produce correct results before deployment on a Hadoop cluster.

1. MRUnit: MRUnit is a testing framework developed for Hadoop MapReduce applications. It allows developers to test Mapper and Reducer logic without running a complete Hadoop cluster.

- Tests Mapper, Reducer, and Driver classes.
- Simplifies debugging and validation.
- Reduces development time.
- Runs locally on a single machine.

2. Unit Testing with MRUnit: Unit testing verifies the correctness of individual Mapper and Reducer functions.

Process

- Create test input data.
- Execute Mapper or Reducer using MRUnit.
- Compare actual output with expected output.

- Identify and fix errors if necessary.

Benefits

- Improves code reliability.
- Detects bugs early.
- Ensures accurate processing.

3. Test Data Preparation: Test data consists of sample input records used to verify MapReduce logic.

Guidelines

- Use representative sample data.
- Include normal and edge-case inputs.
- Prepare expected output for comparison.

- Keep datasets small for quick testing.

Benefits

- Improves test accuracy.
- Helps identify logical errors.
- Simplifies debugging.

4. Local Testing: Local testing executes MapReduce jobs on a single machine without a Hadoop cluster.

Process

- Configure Hadoop in local mode.
- Run MapReduce application.
- Verify output results.
- Debug and optimize code.

Advantages

- Faster execution.
- No cluster required.
- Easier debugging and development.

YARN Framework

YARN (Yet Another Resource Negotiator) is the resource management layer of Hadoop introduced in Hadoop 2. It separates resource management from data processing, allowing multiple applications such as MapReduce, Spark, Hive, and Tez to run on the same Hadoop cluster. YARN improves scalability, cluster utilization, and overall performance.

Objectives of YARN

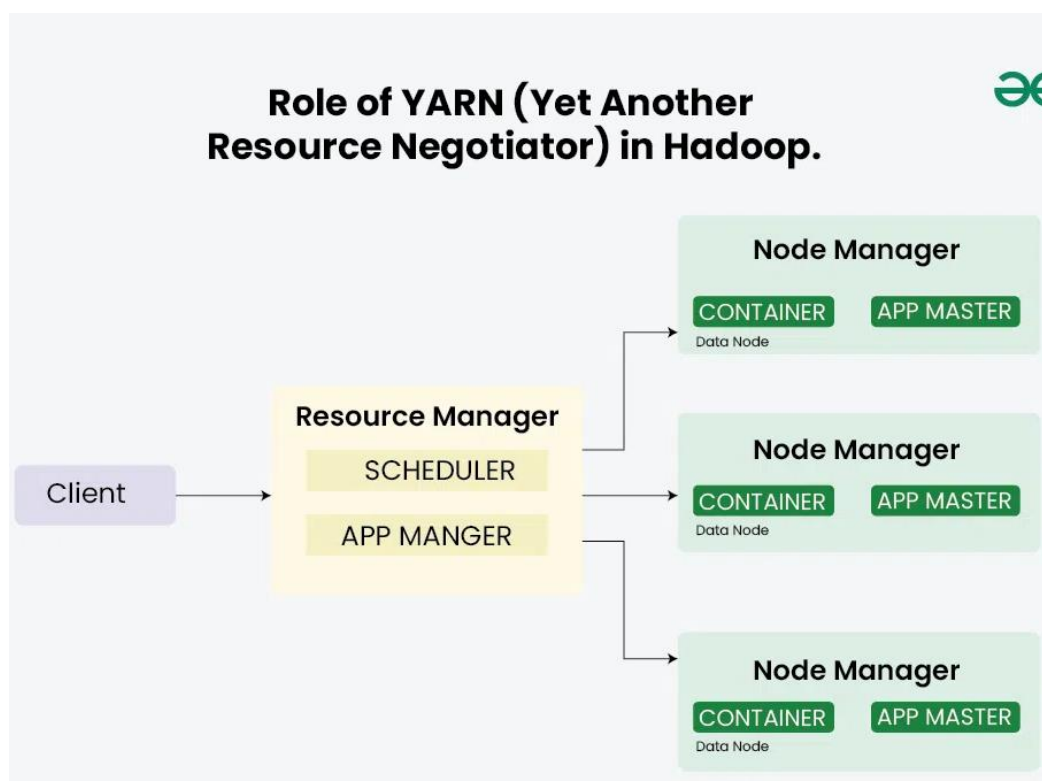
1. Efficient management of cluster resources.
2. Support for multiple data processing frameworks.
3. Improved scalability for large clusters.
4. Better utilization of hardware resources.
5. Simplified job scheduling and monitoring.

Features of YARN

1. Dynamic resource allocation.
2. Multi-tenancy support.
3. High scalability.
4. Fault tolerance.
5. Better cluster utilization.
6. Supports distributed computing applications.
7. Flexible scheduling mechanisms.

Architecture of YARN

YARN follows a master-slave architecture consisting of four main components:



1. ResourceManager (RM)

- Master daemon of YARN.
- Manages cluster-wide resources.
- Allocates resources to applications.
- Performs job scheduling and monitoring.

2. NodeManager (NM)

- Runs on every worker node.
- Manages resources on the local machine.
- Launches and monitors containers.
- Reports node status to ResourceManager.

3. ApplicationMaster (AM)

- Created separately for each application.
- Negotiates resources with ResourceManager.
- Coordinates execution of tasks.
- Monitors application progress and failures.

Advantages of YARN

1. Separates resource management from processing.
2. Supports multiple frameworks beyond MapReduce.
3. Improves scalability and flexibility.
4. Increases cluster utilization.
5. Provides fault tolerance and reliability.
6. Enables efficient resource sharing among applications.

Working of YARN

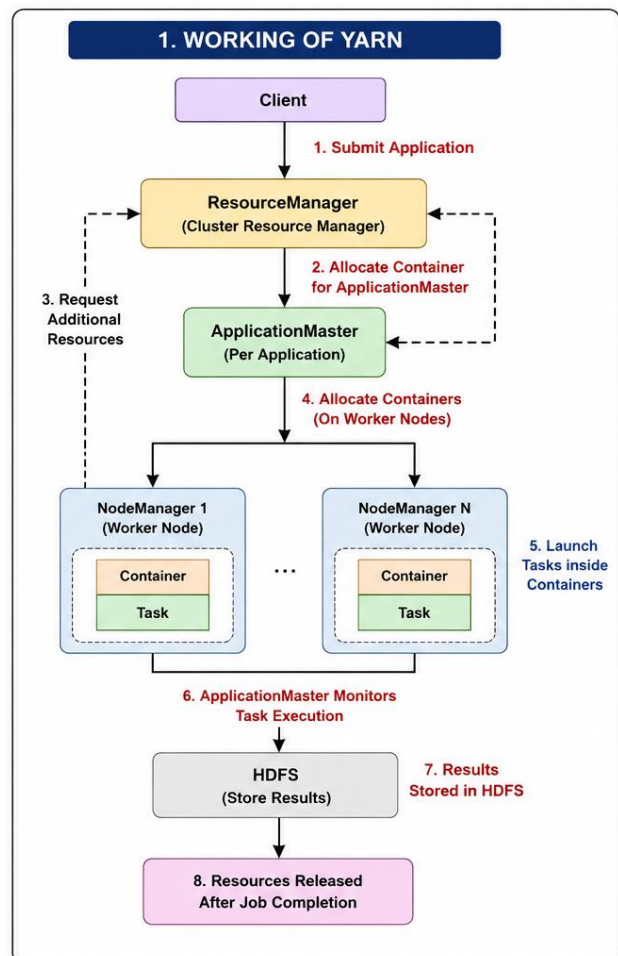
1. Client submits an application to ResourceManager.
2. ResourceManager allocates a container for ApplicationMaster.
3. ApplicationMaster starts and requests additional resources.
4. ResourceManager allocates containers on worker nodes.
5. NodeManagers launch tasks inside containers.
6. ApplicationMaster monitors task execution.
7. Results are generated and stored in HDFS.
8. Resources are released after job completion.

4. Container

- Smallest unit of resource allocation in YARN.
- Contains CPU, memory, disk, and network resources.
- Executes Mapper, Reducer, or other application tasks.

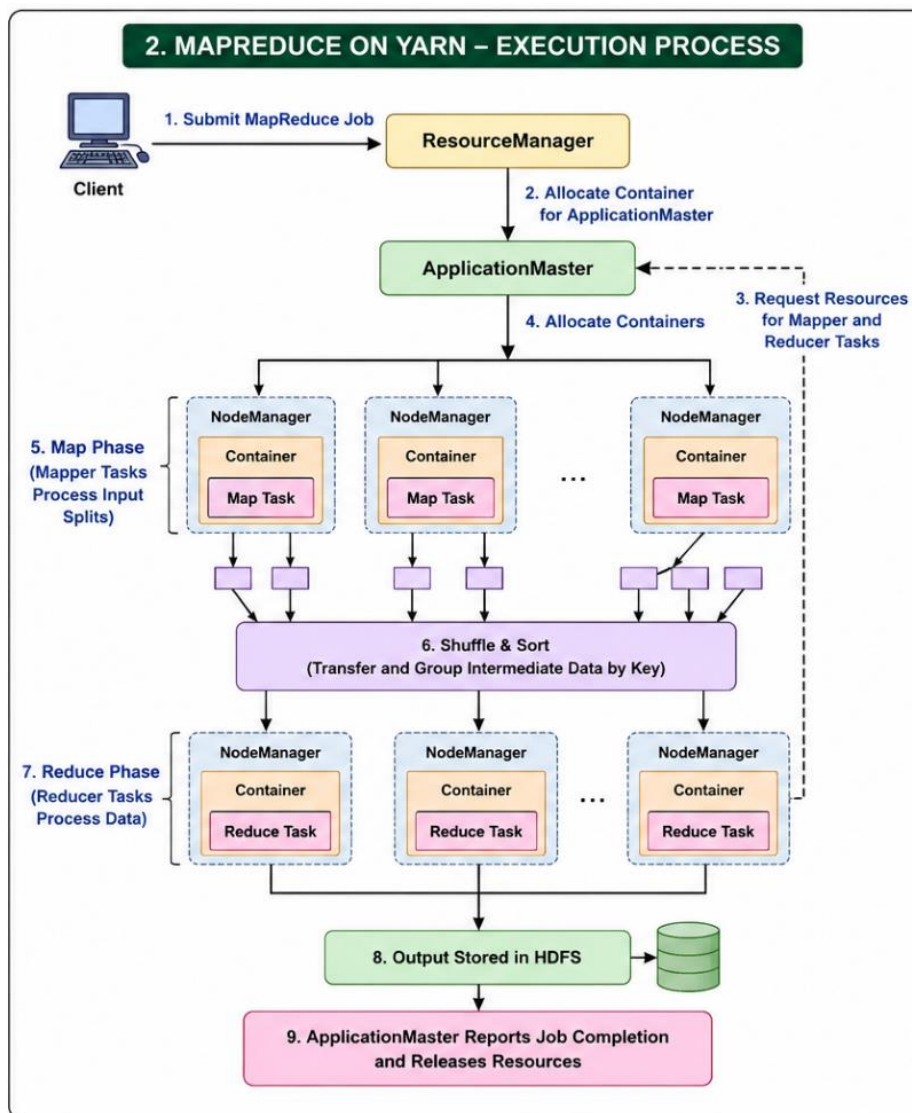
Limitations of YARN

1. Configuration can be complex.
2. Requires additional memory for management services.
3. Monitoring large clusters may be challenging.
4. Performance depends on proper resource allocation.



Execution Process for MapReduce on YARN

1. The client submits a MapReduce job to the ResourceManager.
2. ResourceManager allocates a container for the ApplicationMaster.
3. ApplicationMaster starts and requests resources for Mapper and Reducer tasks.
4. ResourceManager allocates containers on available nodes.
5. NodeManagers launch Mapper and Reducer tasks within containers.
6. Mappers process input data and generate intermediate output.
7. Shuffle and Sort transfer intermediate data to Reducers.
8. Reducers process the data and generate final output.
9. Output is stored in HDFS.
10. ApplicationMaster reports job completion and releases resources.



Failures in MapReduce and YARN

Failures are common in large distributed systems. Hadoop provides mechanisms to detect failures and recover automatically.

1. **Task Failure:** occurs when a Mapper or Reducer task cannot complete successfully.
- Causes
 1. Hardware malfunction.

2. Software bugs.
3. Insufficient memory.
4. Network issues.

Handling

1. Failed tasks are automatically restarted.
2. Tasks may be executed on another node.

2. **Node Failure:** occurs when an entire worker node becomes unavailable.

Causes

1. Power failure.
2. Hardware crash.
3. Operating system failure.
4. Network disconnection.

Handling

1. Heartbeat messages detect node failure.
2. Tasks are reassigned to healthy nodes.
3. HDFS replicas ensure data availability.

3. **Job Failure:** occurs when a complete MapReduce job cannot finish successfully.

Causes

1. Repeated task failures.
2. Application errors.
3. Resource shortages.
4. Configuration problems.

Effects

1. Job execution stops.
2. Error information is logged for analysis.

Recovery Mechanisms

1. **Recovery Mechanisms in MapReduce:** MapReduce includes built-in fault tolerance mechanisms.

Recovery Methods

1. Automatic task re-execution.
2. Data replication through HDFS.
3. Speculative execution for slow tasks.
4. Task reassignment to healthy nodes.

Benefits

1. Improved reliability.
2. Continuous processing despite failures.
3. Reduced data loss.

2. **Recovery Mechanisms in YARN:** YARN provides advanced failure recovery mechanisms.

Recovery Methods

1. ResourceManager monitors cluster health.
2. NodeManager detects local failures.
3. ApplicationMaster can be restarted.
4. Failed containers are recreated automatically.
5. Tasks are rescheduled on available nodes.

Benefits

1. Faster recovery.
2. Better resource utilization.
3. Improved application availability.

Job Scheduling

Job scheduling determines how resources are allocated among multiple jobs in a Hadoop cluster. **Need for Scheduling:** Scheduling ensures efficient utilization of cluster resources and fair execution of jobs.

Objectives

1. Maximize resource utilization.
2. Reduce job waiting time.
3. Improve cluster performance.
4. Ensure fairness among users.

4. **FIFO Scheduler:** FIFO (First In First Out) executes jobs in the order they are submitted.

Features

- Simplest scheduling algorithm.
- Jobs run sequentially based on arrival time.
- Older jobs receive priority.

Advantages

- Easy to implement.
- Low scheduling overhead.

Disadvantages

- Long jobs can delay short jobs.
- Poor resource sharing.

5. **Capacity Scheduler:** Capacity Scheduler divides cluster resources into multiple queues.

Features

- Each queue receives a fixed resource capacity.

- Supports multiple organizations or departments.
- Unused resources can be shared temporarily.

Advantages

- Better resource utilization.
- Supports multi-user environments.
- Prevents resource monopolization.

6. **Fair Scheduler:** Fair Scheduler allocates resources so that all jobs receive a fair share over time.

Features

- Jobs run concurrently.
- Resources are distributed fairly.
- Small jobs finish quickly.

Advantages

- Reduced waiting time.
- Better responsiveness.
- Improved cluster efficiency.

Feature	FIFO	Capacity	Fair
Execution Order	Submission Order	Queue-Based	Fair Sharing
Multi-User Support	No	Yes	Yes
Resource Sharing	Limited	Good	Excellent
Short Job Performance	Poor	Moderate	Good
Complexity	Low	Medium	High

Shuffle and Sort

Shuffle and Sort is the intermediate phase between Mapper and Reducer. It organizes Mapper output before Reducer processing.

7. **Shuffle Process:** transfers intermediate key-value pairs from Mappers to appropriate Reducers.

Steps

1. Mapper generates intermediate data.
2. Data is partitioned based on keys.
3. Intermediate data is transferred across the network.
4. Reducers receive relevant records.

Functions

1. Data transfer.
2. Data grouping.
3. Key distribution.

Importance of Shuffle and Sort

1. Groups records with the same key.
2. Ensures correct reducer input.
3. Improves data organization.

8. **Sorting Process:** Sorting arranges intermediate data by key before it reaches the Reducer.

Steps

1. Received records are grouped by key.
2. Keys are sorted automatically.
3. Sorted data is passed to Reducers.

Benefits

1. Efficient aggregation.
2. Faster reducer execution.
3. Organized data processing.

4. Enables efficient aggregation and analysis.
5. Acts as a bridge between Map and Reduce phases.